

# Vibe Coding Live

Architecture over Syntax

**Ethan Seow** · Practical Cyber x C4AIL  
DEF CON Asia 2026 — AI & AI Security Kampung

# The frame

The most dangerous thing in 2026 is not an AI that writes bad code. It's a human who can't tell when AI wrote bad code.

- My concern isn't capability — Claude Code is genuinely great.
- My concern is engineers who never learned to **reject** a suggestion.
- **Architecture is the discipline of rejection. Syntax is the easy part.**

This is the **Translator capability** in code form: I'm not faster than the AI at typing. I'm faster than the AI at knowing what **should** be typed. That gap is the entire job.

# What you're about to see

A real security tool, built in front of you, in the next eighteen minutes.

- Audience picks the tool. Nothing pre-written.
- Same Claude Code I use on a Tuesday at my desk.
- **At least once, I will reject something Claude writes**, out loud, with reasoning.

That rejection is the whole point. **The code that ships is what's left after I say no.**

## **Fallback prompts on the card:**

- S3 bucket → mask Singapore NRICs → write redacted manifest
- Tail Suricata alerts → de-dupe + enrich → post to Slack
- Classify failed SSH attempts on a honeypot — bot vs targeted human

# Three things I'm watching for

## 1 — Did it make a security decision I should be making?

"Should this be encrypted at rest." "Who can call this endpoint." "What's the failure mode." If AI silently decides those, I roll back.

## 2 — Did it hide a side effect?

AI loves to `os.remove`, `git push`, `subprocess.run` in the middle of a function called `parse_thing`. The function name should equal the function's blast radius. If it doesn't, I rewrite.

## 3 — Did it pretend to handle an error it cannot handle?

`try / except / pass` blocks are the new SQL injection. They turn a loud failure into a silent corruption.

Anything else, I let Claude type. **That's the 98/2 split, applied to my own keyboard.**

# The build cadence

| Min   | Phase              | What I do                                                         | What you watch for                                                 |
|-------|--------------------|-------------------------------------------------------------------|--------------------------------------------------------------------|
| 0-1   | <b>Spec it</b>     | Write 5-line <code>SPEC.md</code> in plain English                | Can I state the tool in 5 lines? If not, I don't understand it yet |
| 1-3   | <b>Skeleton it</b> | Ask Claude for file layout. <b>Reject one suggestion.</b>         | "No <code>utils.py</code> — that's where bugs go to hide"          |
| 3-6   | <b>Tests first</b> | 3 deterministic tests. Claude can suggest; I gate.                | Tests are the spec, compiled                                       |
| 6-11  | <b>Implement</b>   | Claude writes. I read. I narrate one rejection live.              | The planned moment — see next slide                                |
| 11-13 | <b>Run</b>         | Tests red → green. If green-on-first, suspicious — add edge case. | Easy green = missing test                                          |

If **any** phase finishes faster than expected, I add tests. **Time saved is risk surfaced.**

# The planned rejection

~minute 10. Regardless of what Claude writes, I'll find an excuse.

```
# What Claude wrote:
try:
    result = boto3.client('s3').get_object(...)
except Exception:
    pass

# What I replace it with:
try:
    result = boto3.client('s3').get_object(...)
except (BotoCoreError, ClientError) as e:
    logger.error("S3 fetch failed: %s", e)
    raise
```

## Three differences:

1. I named the exceptions I expected.
2. I logged it.
3. I re-raised so the caller can decide.

# Why that rejection matters

## Claude did the intellectual labour

- Figured out which AWS calls to make
- Wired up the SDK
- Generated the boilerplate

## I do the accountability labour

- Decide what happens when the call fails
- Decide what gets logged
- Defend it in the postmortem

`except Exception: pass` is the most dangerous line in modern software. It says "if anything goes wrong I don't care, return success anyway." That decision belongs to the person whose name is on the on-call rotation — not to a probability machine.

# Read > write

What you'll watch most of these eighteen minutes is me **reading**, not typing.

**~80% reading — spec, tests, generated code, diff before commit**

**20% writing**

- Read the spec.
- Read the tests.
- Read every line Claude generated.
- Read the diff before I commit.

**That ratio — read time to write time — is now the entire skill.** An intern would have taken a week for this five years ago. The reason it now takes eighteen minutes is not that AI types faster than the intern. It's that I read faster than the intern.



# This isn't just my framing

Liu, Zhao, Shang, Shen — Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems. arXiv 2604.14228, 14 Apr 2026.

Independent analysis of Claude Code's TypeScript source:

"A simple while-loop that calls the model, runs tools, and repeats... **most of the code, however, lives in the systems around this loop.**"

The five values they extracted from the source:

1. **Human decision authority** ← that's the rejection moment, peer-reviewed
2. Safety / security
3. Reliable execution
4. Capability amplification
5. Contextual adaptability

The tool I'm using right now is itself **98% deterministic harness, 2% model call**. Permission system, context compaction, tool dispatch, session storage — all code. The intelligence is the **edge**. Same architecture I'm asking you to build into your own tools.

# The one habit

If you walk away with one practice from this session:

Before you accept any AI-generated code,  
**say out loud what you would have written instead.**  
If you can't, you don't understand it well enough to ship it.

That's the bar. That's the 98/2. That's the Translator capability in your IDE.

AI's value comes from your data — the spec, the tests, the constraints. Its security comes from your framework — your refusal, your logging, your error model. **Vibe coding doesn't change the fundamentals. It just makes them visible faster.**

# Bridge to Mission 3

Want to feel this in your hands today? **Mission 3 — TDD Speedrun** is the one.

- Same loop: human writes the tests, AI writes the implementation.
- Thirty minutes. Ninety per cent coverage.
- The award goes to the team whose **rejected suggestions** were the most interesting.
- Keep a one-line note every time you say no to Claude.
- **That note is the deliverable.** It's the rubric tiebreaker.

# Close

"I architect. AI types. If you cannot tell me which line you'd reject, you're not vibe coding — you're being driven."

